Báo cáo Workflow: Chuẩn bị dữ liệu, Trích xuất kỹ năng và Chấm điểm phù hợp CV–JD bằng TF-IDF

> **Phạm vi báo cáo.** Đây là báo cáo phần việc của **Đặng Ngọc Quyền** trong dự án nhóm MAT1206E — "CV/Resume Parser and Job Matching System": (1) chuẩn bị dữ liệu, (2) trích xuất kỹ năng từ CV và bản mô tả công việc, (3) tính điểm phù hợp (matching score) và phân loại fit / no-fit. Toàn bộ pipeline chạy bằng Python + scikit-learn, có thể tái tạo từ đầu bằng các lệnh trong mục 6.1.

---

1. Tóm tắt

Dự án nhóm xây dựng một hệ thống so khớp CV (sơ yếu lý lịch) với JD (job description — bản mô tả công việc) để trả lời câu hỏi: *ứng viên này có phù hợp với vị trí này không?* Phần của tôi giải quyết ba bước nền tảng của hệ thống:

1. **Chuẩn bị dữ liệu.** Từ ~94 MB CSV thô (80.017 hàng train), làm sạch, khử trùng lặp, che thông tin cá nhân (PII masking), gán `doc_id` băm md5, dựng bảng cặp (pair) cho 93.733 cặp CV–JD và tách thành kho tài liệu riêng: 643 CV + 351 JD.
2. **Trích xuất kỹ năng.** Dùng **TF-IDF** (term frequency–inverse document frequency — tần suất từ × nghịch đảo tần suất tài liệu, một trọng số đo "từ này quan trọng thế nào trong văn bản này") kết hợp từ điển 7.137 kỹ năng để rút ra danh sách kỹ năng có cấu trúc dạng JSON từ văn bản CV/JD. Kết quả trên tập đánh giá held-out (tập để dành, không dùng khi xây hệ thống): **P@25 = 51.0%** (51% trong 25 kỹ năng trả ra là đúng theo ground truth — đáp án chuẩn), **R@25 = 44.7%** (bắt được 44.7% kỹ năng có trong đáp án), và khi tính trên toàn bộ danh sách trả ra thì **R@full = 74.6%**.
3. **Chấm điểm và phân loại fit.** Với mỗi cặp CV–JD, tính **cosine similarity** (độ tương đồng cosin giữa hai vectơ văn bản) và **skill coverage** (tỷ lệ kỹ năng yêu cầu của JD có mặt trong CV). Trên tập test 13.716 cặp: riêng cosine đã đạt **AUC = 0.805** cho bài toán "Good Fit vs còn lại"; cộng thêm 2 đặc trưng qua **hồi quy logistic** đạt **AUC = 0.810, PR-AUC = 0.443** (gấp ~3.6 lần tỉ lệ dương 12.2% — tức mô hình học được tín hiệu thật); phân 3 vùng (No/Potential/Good) đạt accuracy 63.4% và macro-F1 0.516, vượt baseline đoán toàn "No Fit" (61.7%).

Mọi số liệu đều sinh ra từ script trong repo, có quy trình chọn ngưỡng trên tập train và đo riêng trên tập test, kèm bảng thí nghiệm cho từng quyết định thiết kế (mục 7).

---

2. Bài toán, động lực và phạm vi

2.1 Bài toán

Đầu vào: kho CSV chứa hàng chục nghìn bản ghi — mỗi bản ghi là một đoạn văn CV hoặc JD, thường kèm nhãn quan hệ giữa cặp CV–JD (`No Fit` / `Potential Fit` / `Good Fit`).

Cần đưa ra:

1. **Danh sách kỹ năng có cấu trúc** cho từng CV và từng JD (kỹ năng bắt buộc, kỹ năng ưu tiên, số năm kinh nghiệm, học vấn…), ở dạng JSON — thứ mà các phần sau của nhóm (sắp xếp ứng viên, giao diện) tiêu thụ được.
2. **Một điểm số** cho mỗi cặp CV–JD, để xếp hạng ứng viên và phân loại fit / no-fit.

2.2 Vì sao TF-IDF?

Với phạm vi môn học (nhập môn trí tuệ nhân tạo, không giả định người học đã làm deep learning cho ngôn ngữ) và tài nguyên tính toán, TF-IDF là lựa chọn hợp lý vì:

- **Vật lý được hiểu hết.** Mỗi con số trong pipeline đều tính tay ra được (mục 4.3 có ví dụ tính từng bước). Với mô hình nhúng ngữ nghĩa (embedding như BERT), cần GPU và cột mốc của đồng nghiệp mới so sánh được.

- **Không cần huấn luyện nặng.** TF-IDF chỉ đếm từ trên chính kho dữ liệu (unsupervised — không cần nhãn).
- **Đủ làm nền.** Trích xuất kỹ năng bằng TF-IDF + từ điển đạt R@full ≈ 74.6% trên GT; cosine đạt AUC ≈ 0.81 cho phân loại Good Fit — đều là tín hiệu thật, đo được, có thể dùng làm baseline để đồng nghiệp nâng cấp bằng NER (named entity recognition — nhận diện thực thể) hoặc embedding sau này.

2.3 Phạm vi

- **Trong phạm vi:** làm sạch dữ liệu, dựng cặp, từ điển kỹ năng, trích xuất kỹ năng JSON, đặc trưng điểm (cosine + coverage), hồi quy logistic, đánh giá.
- **Ngoài phạm vi:** phân tích ngữ nghĩa sâu (embedding), học sâu, UI. Phần parser đa mục đích (chat multi-task) của dataset được ghi nhận nhưng bỏ — dùng 172 cặp contact làm ground truth duy nhất cho đồng nghiệp (nêu trong `data/README.md`).

---

3. Bối cảnh và công trình liên quan

- **TF-IDF** ra đời từ họ công trình của Gerard Salton về hệ truy vấn thông tin; khuyến nghị biến thể chuẩn hoá log của tf và công thức idf phổ biến nhất được hệ thống hoá trong Salton & Buckley (1988) [4]. Robertson (2004) [5] lý giải vì sao idf là thước đo "độ ngạc nhiên khi gặp từ" và là nền của BM25. Sách giáo khoa chuẩn là Manning–Raghavan–Schütze, *Introduction to Information Retrieval* (2008), chương 6 [3].
- **Độ đo tương đồng:** cosine similarity là chuẩn của không gian vectơ từ (vector space model) [3]. Jaccard (1901) [6] đưa ra độ đo giao/trên/hợp — tôi dùng cho thí nghiệm so sánh.
- **Đánh giá mô hình nhị phân:** precision/recall/F1 và ma trận nhầm lẫn (confusion matrix) theo Sokolova & Lapalme [9]; ROC-AUC có phân tích chuẩn trong Fawcett (2006) [7]; Davis & Goadrich (2006) [8] chỉ ra khi dữ liệu lệch nhãn (imbalanced — một lớp áp đảo), đường Precision–Recall mô tả chất lượng trung thực hơn ROC — đúng tình huống của dataset này (73% No Fit).
- **Trích xuất kỹ năng:** các công trình NER chuyên sâu như SkillSpan (Zhang et al., 2022 [9']) gắn nhãn kỹ năng trong tin tuyển dụng bằng mô hình transformer; tôi dùng bộ nhãn SkillSpan như **một nguồn vào** cho từ điển kỹ năng (không huấn luyện NER). Từ điển còn ghép từ job-skill-set (batuhanmtl) và ESCO — phân loại kỹ năng chính thức của EU [10].
- **Môi trường tính toán:** scikit-learn [1][2][12][13], pandas, pypdf; toàn bộ chạy CPU thường trong vài phút.

> Chú giải trích dẫn: các ô [n] dẫn đến mục 12. Tài liệu tham khảo; "9'" là SkillSpan nằm cạnh [9] trong danh sách.

---

4. Lý thuyết

Mục này trình bày lý thuyết đầy đủ ký hiệu, theo cách một môn học máy trình bày: định nghĩa → công thức → ví dụ số → liên hệ code.

4.1 Ký hiệu dùng trong toàn bộ báo cáo

Ký hiệu	Ý nghĩa	Ví dụ / giá trị trong dự án
`d`	một tài liệu (document) — một CV hoặc một JD đã làm sạch	`doc_id = c94f1a9af5c3`
`N`	số tài liệu trong kho	994 (643 CV + 351 JD)
`V`	từ vựng (vocabulary) — tập n-gram khác nhau sau khi loại từ hiếm	32.704 n-gram
`t`	một term (term = mục từ) — một từ hoặc n-gram thuộc `V`	`peachtree`, `cpa firm`
`tf(t, d)`	term frequency — số lần `t` xuất hiện trong `d`	tf(peachtree, c94f…)=2
`df(t)`	document frequency — số tài liệu chứa `t`	df(peachtree)=15

Ký hiệu	Ý nghĩa	Ví dụ / giá trị trong dự án
`idf(t)`	inverse document frequency — nghịch đảo tần suất tài liệu	công thức (4.2)
`w(t, d)`	trọng số TF-IDF của `t` trong `d`	công thức (4.3)
`x_d`	vectơ đặc trưng của tài liệu `d`, chiều `|V|`	x_d ∈ ℝ^32704
`‖x‖₂`	chuẩn (norm) L2 của vectơ: `√(Σᵢ xᵢ²)`	= 1 sau chuẩn hoá
`cos(x, y)`	cosine similarity giữa hai vectơ	công thức (4.6)
`S_d`	danh sách kỹ năng đã trích xuất của `d`	25 kỹ năng max
`R_jd`	tập kỹ năng bắt buộc của JD `jd`	31.3 kỹ năng TB
`cov(S, R)`	coverage — tỷ lệ phần tử của `R` có trong `S`	công thức (4.7)
`P, R, F1`	precision, recall, F1-score	công thức (4.10)–(4.12)
`AUC`	diện tích dưới đường ROC	0.805 (cosine, Good)
`t*`	ngưỡng (threshold) tốt nhất theo F1, chọn trên train	0.0639 (cosine)
`ŷ`	nhãn dự đoán; `y` là nhãn thật	y ∈ {No, Potential, Good}
`σ(z)`	hàm sigmoid: `1/(1+e^(-z))`	công thức (4.9)
`θ = (w, b)`	tham số hồi quy logistic	w_cosine = +53.914

Quy ước: chữ `jd` chỉ tài liệu bản mô tả công việc, `cv` chỉ tài liệu sơ yếu lý lịch. "Kỹ năng" (skill) là một cụm từ danh nghĩa chỉ năng lực chuyên môn — ví dụ `sql server`, `communication` (kỹ năng mềm cũng được gắn nhãn như vậy trong ESCO).

4.2 Từ văn bản sang n-gram (tokenization)

Trước khi đếm, văn bản phải tách thành **token** (từ tố — mảnh văn bản nhỏ nhất, ở đây là một từ thường hoặc cụm 2 từ). Quy trình:

1. Chữ thường toàn bộ (`peachtree` ≡ `Peachtree`).
2. Loại dấu và ký tự không phải chữ-số (`sklearn` giữ lại ký tự `+ . # ' -` trong `word` pattern mặc định — nhờ vậy `c++`, `c#`, `asp.net` sống sót).
3. Cắt theo khoảng trắng và dấu câu → token đơn.
4. Ghép **n-gram**: cụm n token liền kề. Tôi dùng `ngram_range=(1, 2)` — cả token đơn lẫn cặp. Lý do: rất nhiều kỹ năng là 2 từ (`sql server`, `machine learning`, `project management`); chỉ dùng token đơn sẽ phá vỡ nghĩa.

Sau đó `min_df = 3` loại mọi term xuất hiện trong ít hơn 3 tài liệu — khử nhiễu đánh máy và tên riêng vô nghĩa, giảm từ vựng từ hàng trăm nghìn xuống 32.704.

4.3 TF-IDF — trọng số từ

Trực giác: một từ quan trọng với một văn bản nếu (a) nó xuất hiện nhiều trong văn bản đó (**tf**) và (b) nó không xuất hiện tràn lan trong mọi văn bản (**idf** — nếu tràn lan thì nó không giúp phân biệt).

Công thức idf (biến thể smooth của scikit-learn [1]):

```
idf(t) = ln( (1 + N) / (1 + df(t)) ) + 1          (4.2)
```

- `ln` là logarit tự nhiên. Cộng 1 vào tử/mẫu ("smoothing") để tránh chia 0 khi df = N, và cộng 1 cuối để idf luôn ≥ 1 (không có term có trọng số âm).
- Ví dụ số trên kho N = 994: từ `peachtree` (phần mềm kế toán, hiếm) có df = 15: `idf = ln(995/16) + 1 = ln(62.19) + 1 = 4.13 + 1 = 5.13`. Từ `experienced` xuất hiện trong 798/994 tài liệu: `idf = ln(995/799) + 1 = 0.219 + 1 = 1.22` — gần 1, tức gần như vô dụng để phân biệt.

Công thức tf (biến thể sublinear của scikit-learn):

```
tf(t, d) = 1 + ln( raw_count(t, d) )   nếu raw_count > 0; ngược lại 0    (4.2a)
```

Dùng log để tần suất tăng 10 lần không làm trọng số tăng 10 lần (từ xuất hiện 100 lần chưa chắc quan trọng gấp 100 lần từ xuất hiện 10 lần).

Trọng số TF-IDF:

```
w(t, d) = tf(t, d) · idf(t)                          (4.3)
```

Ví dụ tính tay, CV `c94f1a9af5c3` (kế toán):

term	raw count	tf = 1+ln(c)	df	idf	w = tf·idf (chưa chuẩn hoá)	w sau chuẩn hoá L2
peachtree	2	1+0.693=1.693	15	5.13	8.68	0.1695
cpa	3	1+1.099=2.099	55	3.88	8.13	0.1588
accounting	7	1+1.946=2.946	310	2.16	6.37	0.1178

Chuẩn hoá L2 (mục 4.4) chia mỗi `w` cho `‖x‖₂ = 7.29` — ba con số cột cuối là kết quả thật in ra từ pipeline (`TfidfVectorizer(sublinear_tf=True, min_df=3, ngram_range=(1,2))`).

Đối chiếu trực giác: `peachtree` (hiếm) thắng `accounting` (phổ biến) dù xuất hiện ít hơn — idf "thưởng" cho tính đặc thù. Còn `experienced`: raw count xuất hiện nhưng chỉ như **tiền tố** trong cụm `experienced professional`, không là token riêng → w = 0.0000. Đây chính là điểm yếu của token hóa theo từ — và là lý do tôi ghép thêm bước khớp từ điển (mục 4.5).

4.4 Chuẩn hoá L2 và cosine similarity

Sau khi tính `w(t, d)` cho mọi term, mỗi tài liệu là một vectơ `x_d ∈ ℝ^|V|`. **Chuẩn hoá L2** chia toàn bộ vectơ cho độ dài của chính nó:

```
x̂_d = x_d / ‖x_d‖₂ ,   ‖x‖₂ = √( Σᵢ xᵢ² )          (4.4)
```

→ mọi vectơ đều nằm trên mặt cầu đơn vị (độ dài = 1). Đo độ giống hai tài liệu bằng **cosine similarity** — cosin của góc giữa hai vectơ:

```
cos(x, y) = (x·y) / (‖x‖‖y‖)                         (4.5)
```

Khi cả hai đã chuẩn hoá L2, mẫu số = 1 nên:

```
cos(x̂, ŷ) = x̂ · ŷ = Σᵢ x̂ᵢŷᵢ                         (4.6)
```

→ chỉ còn một phép nhân-v_cộng (dot product), rất nhanh với ma trận thưa (sparse — ma trận đa số phần tử là 0, chỉ lưu phần tử khác 0). Trong `match_score.py` tôi dùng đúng tính chất này: `C[crow].multiply(J[jrow]).sum(axis=1)` — nhân từng phần tử rồi cộng cột, thu được 93.733 cosine trong một dòng code. Nên nhớ: **cosine bất kể độ dài văn bản** — CV dài 1 trang hay 5 trang đều được "nói cùng một thang" với JD; đó là lý do chọn cosine thay vì dot product thô.

Khoảng giá trị: 0 (không dùng chung term nào) đến 1 (giống hệt). Thực tế trên cặp CV–JD thật: 0.03–0.22 (vì CV và JD nói chuyện khác nhau — JD gọn, CV dài và lan man), trung bình theo nhãn: Good Fit 0.0654 > Potential Fit 0.0503 > No Fit 0.0326 — thứ tự đúng mong đợi.

4.5 Từ điển kỹ năng và quét presence (regex lookaround)

TF-IDF trả ra **term** — nhưng hệ thống cần **skill**. Khác biệt: term là thống kê, skill là đối tượng nghiệp vụ. Cầu nối là một **từ điển kỹ năng (lexicon)** gồm 7.137 cụm — tự gieo hạt từ các khối "Skills" của chính 643 CV (1.146 cụm, lọc

stop-word — từ rác như tên bang Mỹ) + 5.991 cụm từ 3 nguồn ngoài (job-skill-set, SkillSpan, ESCO; trùng nhau 117 cụm đã gộp).

Hai bước dùng từ điển:

(a) Lọc kết quả TF-IDF. Lấy top-200 term điểm cao nhất của mỗi tài liệu, giữ những term nằm trong từ điển, dừng ở 25. Nhờ vậy kết quả luôn là "cụm từ trông như kỹ năng", không phải rác thống kê như `accountant 01` (ngày dính chữ).

(b) Quét presence bằng regex với lookaround. Đếm xem từng kỹ năng từ điển có mặt trong văn bản không — bằng **regex** (biểu thức chính quy — mẫu ký tự để tìm trong chuỗi) với **lookaround** (cơ chế khẳng định "điều kiện quanh vị trí khớp"):

```
(?<![a-z0-9]) KỸ-NĂNG (?![a-z0-9])                (4.5)
```

- `(?<![a-z0-9])` = "trước vị trí này KHÔNG có chữ hoặc số" — ngăn `sql` khớp trong `mysql`.
- `(?![a-z0-9])` = "sau vị trí này KHÔNG có chữ hoặc số" — ngăn `c` khớp trong `c++` sai kiểu.
- Dùng lookaround thay vì `\b` (word boundary — ranh giới từ) vì `\b` **sai** với kỹ năng chứa ký tự đặc biệt: ranh giới giữa `c` và `+` được coi là ranh giới từ, nên `\bc\+\+\b` không khớp đúng. Đây là bài học sửa lỗi thật của dự án (mục 6.6).

Mọi kỹ năng và văn bản đều được **sk_norm** trước khi so: chỉ giữ `[a-z0-9+.#/'-]`, còn lại thay bằng khoảng trắng — nhờ đó `Django/Python` và `django python` khớp nhau.

4.6 Đo độ trùng kỹ năng: coverage

Sau khi có danh sách kỹ năng, độ phù hợp cặp (cv, jd) được đo bằng **coverage** (độ bao phủ):

```
cov(S_cv, R_jd) = | S_cv ∩ R_jd | / | R_jd |       (4.7)
```

- `S_cv` = tập kỹ năng của CV, `R_jd` = tập kỹ năng bắt buộc của JD, `∩` = giao (phần chung).
- `req_cov = 0.8` nghĩa là CV đáp ứng 8/10 kỹ năng bắt buộc.
- Tương tự `pref_cov` cho nhóm kỹ năng ưu tiên (preferred — nice-to-have).

Khớp "phần tử" được định nghĩa theo **n-gram set**: một kỹ năng `s` thuộc `S_cv` ⟺ chuỗi đã `sk_norm(s)` xuất hiện là một cụm 1–4 từ liền trong CV (hàm `ngram_set`, mục 6.5). Cách này đúng biên từ (không khớp `java` trong `javascript`) và tra cứu O(1) bằng frozenset.

Đây là biến thể của ý tưởng Jaccard [6] — tôi chọn coverage thay vì Jaccard vì JD là "bên đặt yêu cầu": nếu CV biết nhiều thứ ngoài yêu cầu (kỹ năng thừa), Jaccard sẽ phạt, coverage thì không — đúng logic tuyển dụng.

4.7 Hồi quy logistic — mô hình học máy tối thiểu

Với 3 đặc trưng `x = (cosine, req_cov, pref_cov)` cần dự đoán nhãn nhị phân `y ∈ {0,1}` (ví dụ y=1 nếu Good Fit). **Hồi quy logistic** (logistic regression) học một đường biên tuyến tính rồi lọc qua hàm sigmoid:

```
z = w₁·cosine + w₂·req_cov + w₃·pref_cov + b        (4.8)
p(y=1 | x) = σ(z) = 1 / (1 + e^(−z))                (4.9)
ŷ = 1  nếu p ≥ 0.5 (hoặc ngưỡng tuỳ chỉnh t*)       (4.9a)
```

- `σ(z)` **sigmoid** nén mọi số thực vào (0, 1) — đọc như xác suất.
- `w`, `b` học bằng **gradient descent** giảm hàm mất mát (loss) — ở đây là **cross-entropy** (mất mát entropy chéo). scikit-learn làm hết; tôi chỉ chọn `class_weight="balanced"` — cộng thêm trọng số nghịch đảo tần suất lớp, để lớp thiểu số (Good Fit, 8%) không bị bỏ rơi [12].
- `t*` (ngưỡng) không cố 0.5: chọn ngưỡng **tối đa hoá F1 trên tập train**, rồi áp nguyên lên test (mục 4.8, hàm `best_f1_threshold`).

4.8 Thước đo đánh giá

Cho bài toán nhị phân, mỗi cặp (thật, dự đoán) rơi vào **ma trận nhầm lẫn** (confusion matrix): TP (true positive — dương đúng), FP (dương giả), FN (âm giả), TN (âm đúng).

```
Precision P = TP / (TP + FP)        "trong những gì ta gắn nhãn dương, đúng bao nhiêu %"
Recall    R = TP / (TP + FN)        "trong toàn bộ dương thật, bắt được bao nhiêu %"
F1 = 2·P·R / (P + R)                trung bình điều hoà của P và R            (4.10–4.12)
```

- **ROC** (receiver operating characteristic — đường đặc trưng hoạt động): vẽ TPR = R theo FPR = FP/(FP+TN) khi quét mọi ngưỡng. **AUC** (area under curve) = diện tích dưới đường — đồng thời là *xác suất mô hình cho một cặp dương-thật điểm cao hơn một cặp âm-thật*. AUC 0.5 = đoán mò, 1.0 = hoàn hảo [7].
- **PR-AUC**: diện tích dưới đường Precision–Recall. Khi dương hiếm (Good Fit chỉ 12.2% tập test), ROC-AUC dễ tạo ảo giác vì TN quá nhiều [8] — nên tôi luôn báo kèm PR-AUC và **prevalence** (tỉ lệ dương trong dữ liệu): PR-AUC 0.443 với prevalence 0.122 nghĩa là mô hình tốt gấp ~3.6 lần ngẫu nhiên.
- **macro-F1**: F1 tính riêng từng lớp rồi lấy trung bình không trọng số — cho thấy chất lượng *trên mọi lớp*, kể cả lớp ít [9].
- **Baseline** (đường nền): mô hình vô dụng nhất hợp lệ — đoán lớp đa số. Accuracy phải thắng baseline mới có nghĩa.

Áp cho bài toán trích xuất kỹ năng: P@K = P trong K kỹ năng đầu trả ra; R@K = R với K đó; R@full = R tính trên toàn bộ danh sách (thường lớn hơn R@K vì quét presence bắt thêm). Ground truth (GT — đáp án chuẩn) là các khối "Skills" có sẵn trong 105 CV held-out, trích bằng cùng bộ quy tắc `parse_skills_block` (đã ghi chú trung thực rằng GT kiểu này có thiên kiến — circular evaluation — nêu ở mục 8).

---

5. Dữ liệu

5.1 Nguồn và các bước tải

Gói	Nguồn	Dùng làm gì
`resume-job-description-fit` (HF: cnamuangtoun)	Hugging Face	CV + JD + nhãn fit chính (dataset đã khai báo trong đề xuất nhóm)
`job-skill-set` (HF: batuhanmtl)	Hugging Face	bổ sung từ điển kỹ năng (4.559 cụm)
`esco_skills_test` (HF: tabiya, nhãn SkillSpan/techwolf)	Hugging Face	bổ sung từ điển (1.445 + 267 cụm)
`resumes-raw-pdf` (HF: d4rk3r) + Kaggle Resume dataset	Hugging Face / Kaggle	**tập test ngoài** 999 CV chưa từng thấy
`job-descriptions` (HF: jacob-hugging-face)	Hugging Face	**tập test ngoài** 100 JD chưa từng thấy

Tải bằng `huggingface_hub` vào thư mục tạm `data/_hf_tmp/` (đã gitignore), ưu tiên bản đã cache cục bộ, bắt lỗi 429 (giới hạn tốc độ của HF với người ẩn danh) rồi fallback — script `data/download_data.py` và `data/build_testset.py`.

5.2 Quy trình làm sạch (build_pairs.py → kho 994 tài liệu)

1. Đọc `matching_train.csv` (80.017 hàng, 698 MB) và `matching_test.csv` (14.321 hàng) — giữ các cột `resume, jd, label, resume_domain, jd_domain`.
2. **Khử trùng lặp tài liệu**: hash toàn văn → chỉ giữ 643 CV và 351 JD duy nhất (những hàng còn lại là *cặp* khác nhau giữa cùng CV/JD — dataset gốc tạo nhãn theo cặp).
3. **Che PII** (personally identifiable information — thông tin nhận diện cá nhân): mọi email → `[EMAIL]`, số điện thoại → `[PHONE]`, URL → `[URL]`. Vừa bảo vệ quyền riêng tư, vừa ngăn mô hình gắn trọng số vào địa chỉ email vô

nghĩa.

4. `doc_id` = md5(toàn văn)[:12] — mã băm 12 ký tự nhận diện ổn định mỗi tài liệu, dùng xuyên suốt (tên file JSON, bảng cặp, demo).
5. Ghi `data/csv/`: `cv_docs.csv` (643), `jd_docs.csv` (351), `pairs.csv` (93.733 cặp: `pair_id, resume_id, jd_id, label, split, resume_domain, jd_domain`).

Nhãn (toàn bộ 93.733 cặp): No Fit 68.774 (73.4%) · Potential Fit 17.489 (18.7%) · Good Fit 7.470 (8.0%) — lệch mạnh, vấn đề sẽ được xử lý ở mục 4.7/4.8.

5.3 Cấu trúc đầu ra trung gian

```
data/
├── extract_json.py            # pipeline chính (trích xuất JSON)
├── improve_skills.py          # thí nghiệm E1–E7 (bằng chứng quyết định thiết kế)
├── match_score.py             # pipeline chấm điểm (bài này mục 6.5)
├── build_pairs.py, build_lexicon_extra.py, build_testset.py, download_data.py
├── skills_lexicon.txt         # 1.146 cụm tự gieo (giữ tay, có thể tái tạo)
├── skills_lexicon_extra.txt   # 5.991 cụm từ nguồn ngoài
├── csv/{cv,jd,pairs}_docs.csv # bảng doc + cặp (gitignore — dữ liệu lớn)
└── extracted/{cv,jd,test_cv,test_jd}/*.json   # 2.093 file JSON kết quả
```

Mỗi file JSON là một tài liệu đã cấu trúc hoá (mẫu thật, CV kế toán `c94f1a9af5c3`):

```
{
  "doc_id": "c94f1a9af5c3",
  "type": "cv",
  "name": null,
  "skills": ["peachtree accounting", "web site", "cpa", "business administration",
             "network", "accountant", "cost reporting"],
  "experience_years": 12,
  "education": {"degree": "bachelor", "major": "accounting",
                "university": "University of Cincinnati"}
}
```

---

6. Triển khai — dạy từng hàm trong code

6.1 Cách chạy (tái tạo toàn bộ từ đầu)

```
python data/download_data.py        # (1) tải dataset gốc từ HF/Kaggle
python data/build_pairs.py          # (2) làm sạch → 643 CV + 351 JD + 93.733 cặp
python data/build_lexicon_extra.py  # (3) dựng từ điển ngoài 5.991 cụm
python data/extract_json.py         # (4) trích xuất JSON + đánh giá held-out
python data/improve_skills.py       # (5) tái tạo bảng thí nghiệm E1–E7
python data/match_score.py          # (6) chấm điểm + phân loại fit + demo
python data/build_testset.py        # (7) dựng 999 CV + 100 JD test ngoài
```

Cần: Python ≥ 3.11, scikit-learn, pandas, numpy, pypdf. Trên Windows chạy `python -X utf8` để in tiếng Việt đúng (mặc định cp1252 gây UnicodeEncodeError).

6.2 `extract_json.py` — pipeline trích xuất (điểm qua, chi tiết trong báo cáo nhóm)

Các hàm chính (số dòng trong file hiện tại):

Hàm (dòng)	Việc	Điểm cốt lõi
`fit` (164)	dựng TF-IDF	`min_df=3, ngram_range=(1,2), sublinear_tf=True`, L2 mặc định
`parse_skills_block` (115)	trích GT từ khối "Skills"	chỉ khối xuất hiện CUỐI cùng; tách theo dấu phẩy/bullet; mục 2–40 ký tự, ≤4 từ; lọc `STOP_SKILLS` (tên bang, llc/inc…)
`build_lexicon` (132)	gieo từ điển	đếm theo **số tài liệu** chứa (không phải số lần), `min_docs≥3`
`dedup_overlap` (170)	khử chứa	bỏ term bị term khác *chứa trọn* (`sql` ⊂ `sql server`) — kiểm tra chứa bằng regex lookaround, không dùng `in` vì biên từ
`topk_of` (184)	chọn term	lấy top-200 điểm ∩ từ điển → dừng ở 25
`extract_skills` (198)	hợp kết quả	`merged = dedup_overlap(top + tier1 + tier2)` ; tier1 = nằm trong vùng 1.500 ký tự sau header "Skills" hoặc dòng bullet (điểm tin cậy cao, xếp trước)
`make_scan_re` (158)	regex quét	sắp xếp các kỹ năng dài→ngắn, lookaround như công thức (4.5)
`cv_name / cv_experience_years / cv_education` (214/229/258)	trường CV	quy tắc nghiêm ngặt, thiếu thì `null` (trung thực hơn đoán bừa)
`jd_title / jd_required_years / jd_split_skills` (287/305/314)	trường JD	tách required/preferred tại từ khoá "preferred/nice to have/bonus"; đếm năm kinh nghiệm trong ngữ cảnh "experience"
`heldout_eval` (361)	đánh giá	in 2 protocol [A]/[B] như mục 7.1

6.3 Từ điển kỹ năng

- **Tự gieo (1.146 cụm):** chạy `parse_skills_block` trên 643 CV, đếm *số tài liệu* chứa mỗi cụm (một CV nhắc 10 lần vẫn chỉ tính 1), giữ `min_docs ≥ 3` → cụm phổ biến trong chính "ngôn ngữ CV" của kho này.
- **Ngoài (5.991 cụm):** job-skill-set + SkillSpan + ESCO label, đã `sk_norm`, loại kỹ năng rác theo cùng `STOP_SKILLS`, trừ trùng với phần tự gieo (117 cụm) và trừ nội bộ (mở rộng đúng mục đích report: P giảm 47.5→43.4 nhưng R tăng 28.1 từ 25.3 — độ phủ rộng hơn).
- **Tổng hợp: 7.137 cụm** — tự gieo 1.146 + ngoài 5.991.

6.4 Vì sao không dùng `in` khi khớp kỹ năng

`"sql" in "mysql server"` → True (SAI về nghiệp vụ). Hai cách đúng:

1. **Regex lookaround** `(?<![a-z0-9])sql(?![a-z0-9])` — dùng khi quét trên văn bản thô, nhanh vì hợp các kỹ năng thành một regex khổng lồ, quét 1 lượt.
2. **n-gram set** — dùng khi so *danh sách với danh sách* (coverage): dựng mọi cụm 1–4 từ của CV thành frozenset, tra `O(1)` (hằng — không phụ thuộc kích thước). Khớp `sql server` ✓, không khớp `sql server 2019` (4+ từ) — chấp nhận được vì dạng có năm không phải skill.

6.5 `match_score.py` — pipeline chấm điểm (dạy chi tiết)

Bước 1 — nạp và ánh xạ (hàm `load_all`, `row_of`). Đọc 994 tài liệu + 93.733 cặp. Với mỗi cặp, `crow = map_cv[resume_id]`, `jrow = map_jd[jd_id]` dịch từ mã băm sang chỉ số dòng trong ma trận — phép tra bảng O(1).

Bước 2 — TF-IDF dùng lại (không fit lại). Gọi đúng `fit` của `extract_json.py` trên 994 tài liệu, thu `X` (994 × 32.704, L2 chuẩn hoá) → cắt `C = X[:643]` (CV) và `J = X[643:]` (JD). Vì hai ma trận cùng không gian term, cosine giữa chúng hợp lệ. Đây là điểm hay của việc tách module: *không sao chép code fit*, nên vectơ dùng để chấm điểm **giống hệt** vectơ đã dùng để trích xuất.

Bước 3 — cosine 93.733 cặp trong một dòng:

```
cosine = np.asarray(C[crow].multiply(J[jrow]).sum(axis=1)).ravel()   # công thức (4.6)
```

Đọc từ trong ra: `C[crow]` xếp lại 643 hàng CV thành ma trận 93.733 hàng (mỗi cặp 1 hàng); `.multiply(J[jrow])` nhân từng phần tử (dot product rải theo hàng); `.sum(axis=1)` cộng cột → mỗi hàng 1 số; `.ravel()` phẳng thành mảng 1 chiều. Với ma trận thưa của scipy, `.multiply` giữ dạng thsparse — 93.733 × 32.704 phần tử chỉ còn ~8 triệu phần tử khác 0 trong RAM.

Bước 4 — coverage (hàm `ngram_set` + `build_features`).

```
def ngram_set(text, max_k=4):
    w = sk_norm(text).split()                    # chữ thường, giữ + . # / ' -
    out, n = set(), len(w)
    for i in range(n):
        for k in range(1, max_k + 1):
            if i + k <= n:
                out.add(" ".join(w[i:i+k]))      # mọi cụm 1..4 từ liền kề
    return frozenset(out)                        # frozenset = set không đổi, băm được
```

Với JD có `required_skills` → `req_cov = |R ∩ S_cv| / |R|`; JD không có danh sách (31.3 kỹ năng là trung bình, nhưng 33% JD không có trường năm kinh nghiệm) → đặt **NaN** (không phải 0! — 0 nghĩa là "CV không biết gì", NaN nghĩa là "không biết vì thiếu dữ liệu") và các thước đo bỏ qua NaN bằng mask.

Bước 5 — chọn ngưỡng (hàm `best_f1_threshold`).

```
def best_f1_threshold(y, score):
    cands = np.unique(np.quantile(score[np.isfinite(score)],
                                  np.linspace(0.01, 0.99, 197)))
    best = (0.0, 0.0)
    for t in cands:
        f1 = f1_score(y, score >= t, zero_division=0)
        if f1 > best[1]:
            best = (t, f1)
    return best
```

Quét 197 ngưỡng (quantile 1%..99% của chính điểm số), tính F1 từng ngưỡng, lấy max. Chỉ chạy **trên train** — giá trị trả về `t*` được *đóng băng* rồi mới áp lên test. Nếu chọn ngưỡng trên test thì là rò rỉ dữ liệu (data leakage — dùng đáp án để làm bài), kết quả sẽ đẹp giả tạo.

Bước 6 — báo cáo (hàm `binary_report`, `band_report`). `binary_report` in AUC, PR-AUC, và P/R/F1 tại `t*`; `band_report` chia điểm thành 3 vùng `[0, t_lo) / [t_lo, t_hi) / [t_hi, 1]` → in ma trận nhầm lẫn 3×3, accuracy, macro-F1. Ngưỡng 3 vùng cũng chọn trên train: `t_lo` từ bài "Fit vs No Fit", `t_hi` từ bài "Good vs Potential".

Bước 7 — hồi quy logistic.

```
lr = LogisticRegression(max_iter=2000, class_weight="balanced")
lr.fit(Xtr, ytr)                    # Xtr = 80.017 cặp train, 3 cột đặc trưng
p = lr.predict_proba(Xte)[:, 1]     # xác suất lớp dương trên 13.716 cặp test
```

`class_weight="balanced"` = trọng số lớp ∝ 1/tần suất → Good Fit (8%) được phạt gấp ~9 lần khi bỏ sót so với No Fit (73%) — đối trọng cho dữ liệu lệch [12].

Bước 8 — demo (ghi `demo_matches.json`). Rút 3 cặp thật có điểm cao nhất từng nhãn từ test, truy ngược JSON gốc của CV/JD, in ra *từng kỹ năng khớp / thiếu* — bằng chứng pipeline không chỉ cho ra con số mà còn giải thích được (mục 7.5).

Bước 9 — ghi `data/scored/pairs_scored.csv` — 93.733 hàng `(pair_id, resume_id, jd_id, label, split, cosine, req_cov, pref_cov)` — đầu vào cho phần xếp hạng của đồng nghiệp.

6.6 Bẫy đã gặp và bài học (ghi để sau này đỡ tốn thời gian lại)

1. `\b` **sai với** `c++`, `.net`, `c#` — ranh giới từ của regex tính ký tự đặc biệt là ranh giới; đổi sang lookaround `(?<![a-z0-9])…(?![a-z0-9])` (mục 4.5).
2. **NaN vs 0 cho đặc trưng thiếu** — điền 0 thì mô hình hiểu "CV không biết gì"; phải để NaN + mask khi đánh giá.
3. `python` **không phải** `python -X utf8` — Windows cp1252 ném UnicodeEncodeError khi in tiếng Việt; cố định chạy `-X utf8`.
4. **workdir của pwsh có thể bị bỏ qua** — mọi script dùng `os.path.join(DATA, …)` tương đối và phải chạy từ thư mục gốc dự án.
5. **Kỹ năng trùng lặp trong output** — `dedup_overlap` chỉ bỏ term bị *chứa*, không bỏ term *giống hệt*; hợp kết quả phải qua `sorted(set(…))`.
6. **Rò rỉ khi fit vectorizer trên cả test** — mọi `fit` đều trên phần train/994 doc chính; tập ngoài chỉ `transform`/quét presence.
7. **Số liệu phải ghi rõ protocol** — cùng một mô hình cho 3 con số khác nhau nếu: (a) chấm chỉ top-25 hay cả danh sách, (b) exact hay containment, (c) ngưỡng chọn trên train hay test. Mọi bảng kết quả dưới đây đều ghi protocol ngay trên tiêu đề.

---

7. Kết quả và thảo luận

7.1 Trích xuất kỹ năng — đánh giá held-out (129 CV, 105 có GT)

Hai protocol song song, cùng 514 train / 129 held-out, seed 42, TF-IDF fit trên 514 CV train:

- **[A] Protocol cũ** — chỉ chấm 25 kỹ năng top điểm TF-IDF, khớp *exact* từng chuỗi với GT.
- **[B] Protocol thật** — chấm đúng *đầu ra cuối cùng* của hệ thống (top + tier + quét presence, đã dedup), khớp theo *chứa trọn biên từ* (containment).

Protocol	P@25	R@25	R@full
[A] cũ (top-25, exact)	43.4%	28.1%	—
[B] mới (output thật, containment)	**51.0%**	**44.7%**	**74.6%**

Thảo luận. Nhảy vọt từ [A] sang [B] không phải do "chăm hơn" mà do 3 quyết định có bằng chứng (mục 7.2): khớp containment (+11.6 điểm R@25), sắp xếp tier (lucky-first — đưa kỹ năng nằm gần khối "Skills" lên đầu, +3.8 P), và hygiene từ điển (loại tên bang, tăng min_docs). Lưu ý protocol cũ đánh giá *sai đối tượng* (chỉ top-25 TF-IDF, không phải output thật) — bài học về việc "đo đúng thứ mình đóng gói".

7.2 Bảng thí nghiệm — mỗi quyết định một hàng (protocol [B], held-out)

#	Cấu hình	P@25	R@25	R@full	Kết luận
0	baseline: chỉ top-25 TF-IDF, exact	43.4	28.1	—	mốc so sánh
E1	output thật (top + presence scan), exact	41.6	29.7	54.9	quét presence tăng recall nhưng exact phạt dư từ
E2	+ khớp containment	47.3	39.7	74.6	+11.6 R@25 — biến thể viết khác được nhận diện
E3	+ tier ordering (vùng "Skills"/bullet lên đầu)	**51.0**	**44.7**	74.6	+3.8 P — Lucky-first đúng cơ chế
E4	+ bình hoá số nhiều (databases→database)	50.2	45.4	76.1	không đáng — P giảm

#	Cấu hình	P@25	R@25	R@full	Kết luận
E5	+ lọc skill generic (df > 0.5)	50.5	44.3	64–68	**LOẠI** — skill generic vẫn có trong GT
E6	hạ min_docs 3→2 (+544 cụm)	50.5	45.2	79.5	**LOẠI** — P giảm, nhiễu tăng
E7	+ từ điển từ CV test	50.1	45.3	76.1	**LOẠI** — không giá trị, rò rỉ rủi ro

→ **Cấu hình cuối: E3.** Mọi thí nghiệm dùng đúng protocol held-out (chọn trên train, đo trên held-out), script `data/improve_skills.py` tái tạo được từng hàng.

7.3 Độ phủ ngoài (tài liệu chưa từng thấy — kiểm tra tổng quát hoá)

Tập	N	≥1 skill	≥5	≥10	Số skill TB	zero
CV chính	643	100%	100%	100%	76.0	0
JD chính	351	100%	99.1%	95.2%	39.7	0
CV test ngoài	999	94.3%	73.2%	57.8%	26.5	57
JD test ngoài	100	100%	99%	94%	37.3	0

- 57/999 CV test ngoài cho 0 kỹ năng: kiểm tra trung bình 1.237 ký tự so với 3.820 ký tự của CV có kết quả — phần lớn là PDF trích xuất được quá ít văn bản (giới hạn của pypdf), không phải lỗi pipeline. Điều này được ghi rõ trung thực trong `data/README.md`.
- Trường khác trên CV chính: degree 99.7%, major 62.1%, university 93.0%, experience_years 22.2%, name 0% (quy tắc nghiêm ngặt — không đoán bừa tên); JD: title 51.0%, required trung bình 31.3 kỹ năng, preferred 8.4, years 66.7%.

7.4 Chấm điểm & phân loại fit — tập test 13.716 cặp (ngưỡng chọn trên train)

Thống kê theo nhãn (toàn bộ dữ liệu — mô tả tính chất tín hiệu):

	cosine TB	req_cov TB
Good Fit	0.0654	0.2615
Potential Fit	0.0503	0.2198
No Fit	0.0326	0.1425

→ Cả hai đặc trưng tăng đơn điệu theo "độ phù hợp" — tiền đề cho mô hình học.

Bài nhị phân (test 13.716 cặp; prevalence Good = 12.2%, No = 61.7%):

Mô hình	Đối tượng	AUC	PR-AUC	P	R	F1
chỉ cosine	Good vs còn lại	0.805	0.433	0.464	0.424	0.443
chỉ req_cov	Good vs còn lại	0.757	0.310	0.298	0.466	0.363
LR (3 đặc trưng)	Good vs còn lại	**0.810**	**0.443**	0.441	0.470	0.455
LR (3 đặc trưng)	Potential vs còn lại	0.647	0.343	0.332	0.770	0.464
LR (3 đặc trưng)	No Fit vs còn lại	0.761	0.817	0.699	0.929	0.798

Hệ số LR (lớp Good): `cosine +53.914 · req_cov +0.471 · pref_cov +0.172 · b −2.689` — cosine chi phối (đặc trưng gốc ở thang nhỏ), req_cov vẫn đóng góp dương; dấu tất cả đúng trực giác.

Bài 3 vùng (No / Potential / Good) — ngưỡng `t_lo = 0.0459, t_hi = 0.0639` chọn trên train:

```
                dự đoán →   No     Potential   Good
No Fit  (8.461)          6.943    1.246       272
Potential(3.582)         1.993    1.040       549
```

```
Good Fit (1.673)          486      477       710
accuracy 0.634 · macro-F1 0.516 · baseline đoán toàn "No Fit" = 0.617
```

Thảo luận:

- *Cosine đơn lẻ gần bằng LR 3 đặc trưng* (0.805 vs 0.810 AUC): tín hiệu chủ lực là trùng từ vựng; coverage chỉ cải thiện nhẹ. Điều này hợp lý vì nhãn của dataset này được tạo từ chính độ trùng nội dung (kiểm chứng: cùng JD `41db8fe628ff`, 3 CV có nhãn No/Potential/ Good có cosine 0.117/0.179/0.219 tăng đều — mục 7.5).
- *Lớp Potential khó nhất* (AUC 0.64-0.65): nằm giữa, ranh giới mờ — kể cả với con người. Đây là giới hạn của nhãn 3 lớp với đặc trưng bề mặt.
- *PR-AUC là thước đo trung thực ở đây*: Good Fit chỉ 12.2% — PR-AUC 0.443 = 3.6× ngẫu nhiên; nếu chỉ nhìn ROC-AUC 0.81 dễ ảo tưởng vì TN áp đảo.
- *Trên thực tế dùng 2 lớp + xếp hạng*: dùng cosine làm điểm xếp hạng ứng viên trong từng JD, lấy `t_lo` làm cổng lọc no-fit — đủ cho phần sau của nhóm; 3 vùng chỉ để tham chiếu.

7.5 Demo — giải thích được (3 cặp thật từ `demo_matches.json`, cùng JD "Java web dev")

	CV abe506a3b114 (Good)	CV b4f87f0183e5 (Potential)	CV e06e1b4e5f42 (No)
cosine	0.2187	0.1789	0.1171
req_cov	0.800	0.867	0.667
khớp (rút gọn)	angular, java, j2ee, ajax, jquery, json, css, html…	java, jsp, j2ee, xml, ajax… (thiếu **angular**)	java, javascript, css, xml… (thiếu **angular, j2ee, json**)
thiếu	bootstrap, jsp, xml	angular, bootstrap	angular, bootstrap, jsp, j2ee, json

Nhận xét có giáo dục: cặp "No Fit" vẫn khớp 10/15 kỹ năng — nhãn không phải hàm của coverage một cách máy móc (khác biệt nằm ở ngữ nghĩa/phạm vi công việc mà TF-IDF không thấy). Pipeline in ra *danh sách khớp/thiếu* → con người soát được từng quyết định.

---

8. Giới hạn

1. **Circular evaluation (GT vòng).** GT trích xuất kỹ năng là khối "Skills" trong chính CV, trích bằng cùng bộ quy tắc — thuận cho phương pháp của tôi. R@full 74.6% là *thiên tích cực* trên GT này, nhưng *thiên tiêu cực* với GT hẹp (GT chỉ ghi ~3,588 cụm trên 105 CV, trong khi CV nhắc nhiều hơn). Kết luận an toàn: so sánh **tương đối** giữa các cấu hình là đáng tin; con số tuyệt đối đọc kèm "±".
2. **Nhãn fit do dataset tự tạo**, phân bố lệch (73/19/8), ranh giới Potential mờ — mô hình học giỏi tối đa bằng chất lượng nhãn (garbage in, garbage out).
3. **Không có ngữ nghĩa.** `java` (ngôn ngữ) và `javascript` là 2 term khác nhau — với TF-IDF đúng vậy; nhưng "developer" vs "engineer" thì không nối được. NER/embedding của đồng nghiệp là hướng nâng cấp trực tiếp.
4. **PDF hỏng làm sai lệch test ngoài** (57 CV zero — văn bản gốc quá ít).
5. **JD thiếu trường** (title 51%, years 67% trên JD chính) — quy tắc trả `null` thay vì đoán; phần sau cần xử lý null.
6. **Chỉ 643 CV/351 JD chính** — 80.017 hàng phần lớn là cặp trùng tài liệu; thống kê mô hình chấm điểm dựa trên 93.733 cặp nhưng từ 994 tài liệu gốc.

---

9. Thay đổi so với đề xuất

- Đề xuất cột mốc "trích xuất kỹ năng TF-IDF" → **hoàn thành và mở rộng**: từ khái niệm ban đầu (TF-IDF xếp hạng term) phát triển thành pipeline 3 tầng (TF-IDF → từ điển → tier+quét presence) sau khi thí nghiệm chứng minh mỗi tầng đóng góp riêng (mục 7.2).

- Đề xuất "tính điểm phù hợp + matched/missing skills" → **hoàn thành** (mục 7.4–7.5), đầu ra `pairs_scored.csv` sẵn sàng cho phần xếp hạng.
- Bổ sung ngoài đề xuất: bộ thí nghiệm có kiểm soát (improve_skills.py), tập test ngoài 1.099 tài liệu, hướng dẫn giảng dạy 20 trang (`docs/bai-giang-trich-xuat-skill-tfidf.pdf`).

---

10. Đóng góp, sử dụng AI và tài nguyên bên ngoài

- **Đóng góp cá nhân (tôi):** toàn bộ phần dữ liệu, trích xuất kỹ năng, chấm điểm trong báo cáo này — 7 script Python (~2.000 dòng), 2 từ điển, 2.093 file JSON, bảng thí nghiệm, tài liệu hướng dẫn; các phần khác (parser đa mục đích, NER, giao diện) của đồng nghiệp.
- **AI:** dùng trợ lý lập trình để soạn/hiệu chỉnh code và tài liệu; mọi quyết định thiết kế, thí nghiệm và số liệu do tôi kiểm soát, script đều chạy được độc lập (không có bước "ma thuật" không tái tạo được).
- **Tài nguyên:** dataset và từ điển liệt kê ở mục 5.1; không dùng trọng số mô hình học sâu nào; toàn bộ chạy CPU cục bộ.

---

11. Kết luận

Phần của tôi đưa dự án từ CSV thô 80.017 hàng đến trạng thái "sẵn sàng cho phần còn lại của hệ thống": 994 tài liệu sạch có cấu trúc JSON, 7.137 kỹ năng từ điển, điểm cosine + coverage cho 93.733 cặp (AUC 0.81 cho Good-vs-rest, vượt baseline rõ rệt, giải thích được từng kỹ năng khớp/thiếu), và một bộ đánh giá trung thực (2 protocol, ngưỡng đóng băng trên train, PR-AUC kèm prevalence). Hướng phát triển tự nhiên: NER/embedding cho kỹ năng (đồng nghiệp), xếp hạng theo cặp với learning-to-rank, và gán GT thủ công cho ~50 JD để đo công bằng thêm một chiều.

---

12. Tài liệu tham khảo

[1] Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." *JMLR* 12 (2011): 2825–2830. — https://scikit-learn.org/stable/ (truy cập 2026-11-04)

[2] scikit-learn, "Text feature extraction" (User Guide §6.2.3: sublinear tf, smooth idf, n-grams, L2 normalisation). — https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction (truy cập 2026-11-04)

[3] Manning, C., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008, ch. 6 (term weighting, vector space model, cosine).

[4] Salton, G., Buckley, C. "Term-weighting approaches in automatic text retrieval." *Information Processing & Management* 24(5) (1988): 513–523.

[5] Robertson, S. "Understanding inverse document frequency: on theoretical arguments for IDF." *Journal of Documentation* 60(5) (2004): 503–520.

[6] Jaccard, P. "Étude comparative de la distribution florale dans une portion des Alpes et du Jura." *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901): 547–579.

[7] Fawcett, T. "An introduction to ROC analysis." *Pattern Recognition Letters* 27(8) (2006): 861–874.

[8] Davis, J., Goadrich, M. "The relationship between Precision-Recall and ROC curves." *Proc. ICML* (2006): 233–240.

[9] Sokolova, M., Lapalme, G. "A systematic analysis of performance measures for classification tasks." *Information Processing & Management* 45(4) (2009): 427–437.

[10] Zhang, M., Jensen, K.N., Dol, S., Sequeira, M.V.d., Plank, B. "SkillSpan: Hard and Soft Skill Extraction from English Job Postings." *NAACL-HLT* (2022). arXiv:2202.12271. — https://arxiv.org/abs/2202.12271 (truy cập 2026-11-04)

[11] European Commission, **ESCO** — European Skills, Competences, Qualifications and Occupations. — https://esco.ec.europa.eu/ (truy cập 2026-11-04)

[12] Dataset (HF Datasets, truy cập 2026-11-04): `cnamuangtoun/resume-job-description-fit` (CV+JD+nhãn chính) · `batuhanmtl/job-skill-set` (từ điển) · `tabiya/esco_skills_test` (SkillSpan/techwolf) · `d4rk3r/resumes-raw-pdf` (CV test ngoài) · `jacob-hugging-face/job-descriptions` (JD test ngoài) · Kaggle `sachinkelenjaguri/resume-dataset` (CV test ngoài bổ sung).

[13] scikit-learn API (truy cập 2026-11-04): `TfidfVectorizer` — https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html · `LogisticRegression` (class_weight="balanced") — https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html · `metrics` (f1_score, roc_auc_score, precision_recall_curve) — https://scikit-learn.org/stable/modules/model_evaluation.html

[14] Công cụ: pandas (https://pandas.pydata.org/), numpy (https://numpy.org/), pypdf (https://pypdf.readthedocs.io/), Python 3.13 (https://www.python.org/) — truy cập 2026-11-04.